

CO3 AT2 - Question 1

Case Study: Airline Reservation Sorting System (Merge Sort)

Aim:

To analyze how Merge Sort ensures stable and efficient sorting in an airline reservation system based on departure time and priority.

Problem Statement:

An airline reservation system sorts flight bookings based on departure time and priority. Analyze how Merge Sort ensures stable and efficient sorting. Identify issues such as large data volume and real-time updates. Apply divide and conquer techniques to explain the process. Analyze time complexity and design a solution, justifying its effectiveness.

Objectives:

- Implement Merge Sort for sorting flight bookings.
- Analyze the stability and efficiency of Merge Sort.
- Study the impact of large datasets and real-time updates.
- Apply Divide and Conquer strategy in the sorting process.

Divide and Conquer Approach:

1. Divide the booking records into smaller sublists.
2. Recursively sort each sublist.
3. Merge the sorted sublists to obtain the final sorted list.

Issues Identified:

- Large volume of booking records.
- Frequent real-time updates and cancellations.
- Requirement to preserve order among equal departure times.

Why Merge Sort?

Merge Sort is a stable sorting algorithm. If two bookings have the same departure time, their original order is preserved after sorting. This property is essential in airline reservation systems where booking priority must be maintained.

Time Complexity:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Result and Analysis:

The airline booking records were successfully sorted using Merge Sort. The algorithm efficiently handled booking data while preserving the order of equal elements. The divide and conquer strategy ensured consistent $O(n \log n)$ performance even for large datasets.

Conclusion:

Merge Sort is highly suitable for airline reservation systems because it guarantees stable sorting and predictable performance. It effectively handles large datasets while maintaining booking priority and departure order.

Output:

(Insert program output screenshot here.)